Python Operator

In Python programming, Operators in general are used to perform operations on values and variables.

Python provides the following set of operators

1) Arithmetic Operators
2) Relational Operators OR Comparison Operators
3) Logical operators
4) Bitwise operators
5) Assignment operators
6) Special operators

Arithmetic Operators

1) + → Addition

2) – → Subtraction

3) * → Multiplication

4) / → Division Operator

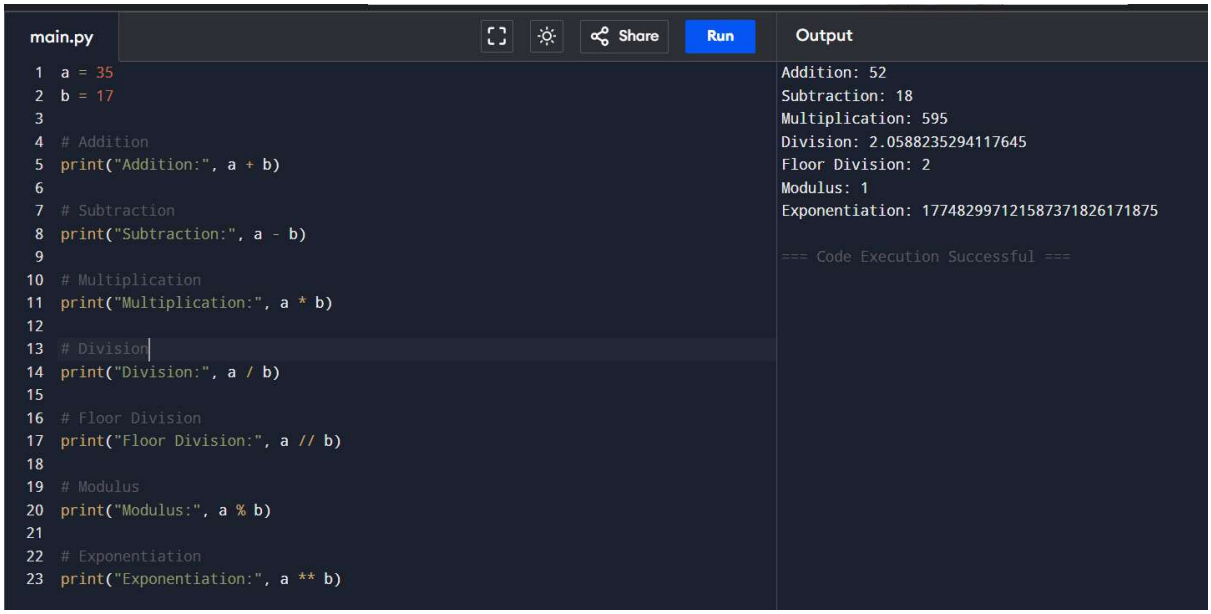
5) % → Modulo Operator

6) // → Floor Division Operator

7) ** → Exponent Operator OR Power Operator

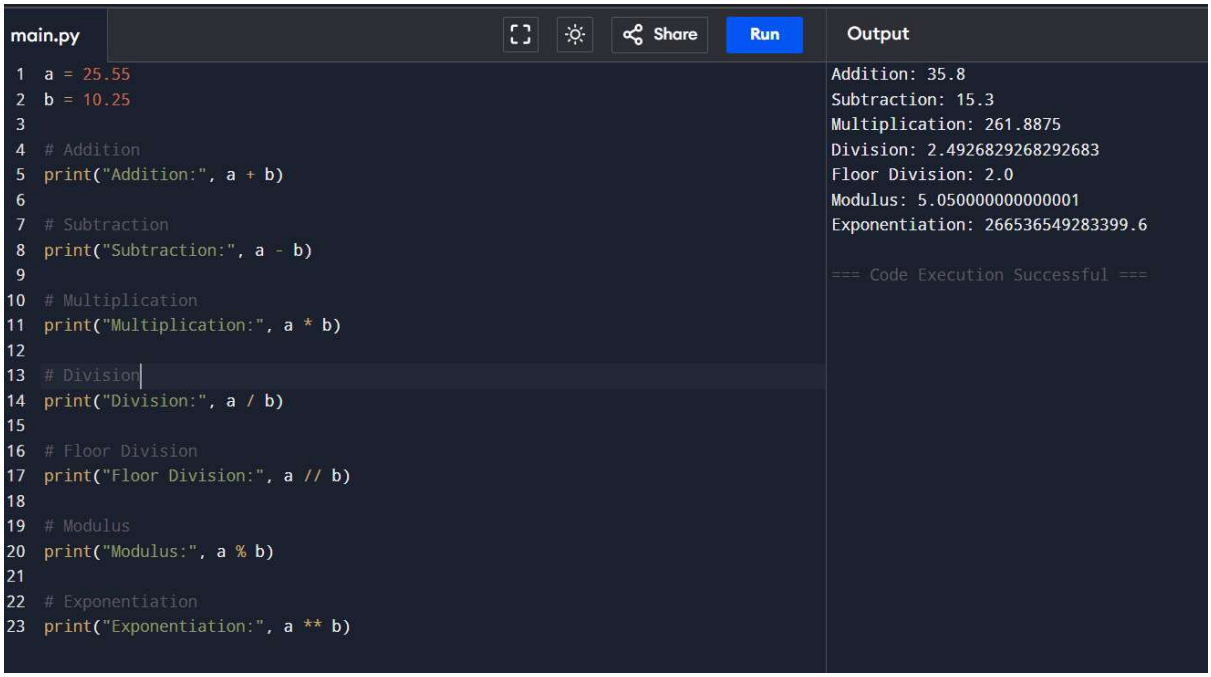
Example 1:-

```
a = 35
b = 17

# Addition
print("Addition:", a + b)

# Subtraction
print("Subtraction:", a - b)

# Multiplication
print("Multiplication:", a * b)

# Division
print("Division:", a / b)

# Floor Division
print("Floor Division:", a // b)

# Modulus
print("Modulus:", a % b)

# Exponentiation
print("Exponentiation:", a ** b)
```



Example 2:-

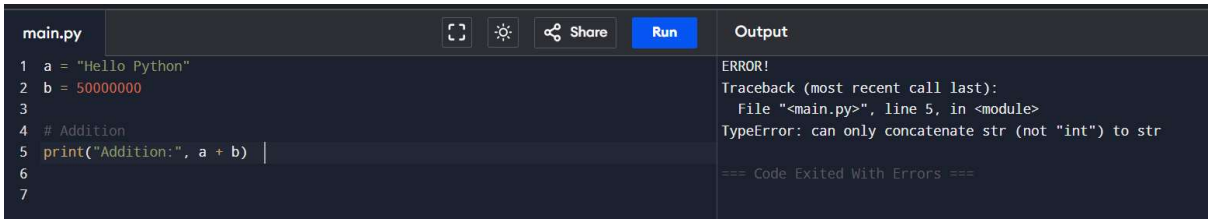
```
a = 25.55
b = 10.25

# Addition
print("Addition:", a + b)

# Subtraction
print("Subtraction:", a - b)

# Multiplication
print("Multiplication:", a * b)

# Division
print("Division:", a / b)

# Floor Division
print("Floor Division:", a // b)

# Modulus
print("Modulus:", a % b)

# Exponentiation
print("Exponentiation:", a ** b)
```



Example 3:-

a = "Hello Python"
b = 50000000

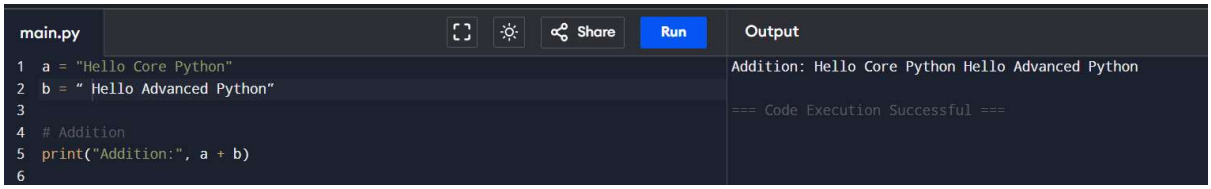
 # Addition
print("Addition:", a + b)



Example 4:-

a = "Hello Core Python"
b = “Hello Advanced Python”

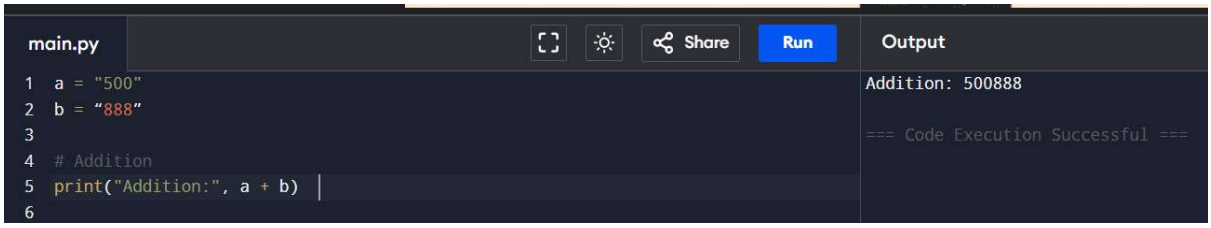
 # Addition
print("Addition:", a + b)



Example 5:-

a = "500"
b = “999

 # Addition
print("Addition:", a + b)



Ques:

What is the difference between Division & Floor Division Operator?

/ operator always returns float value.

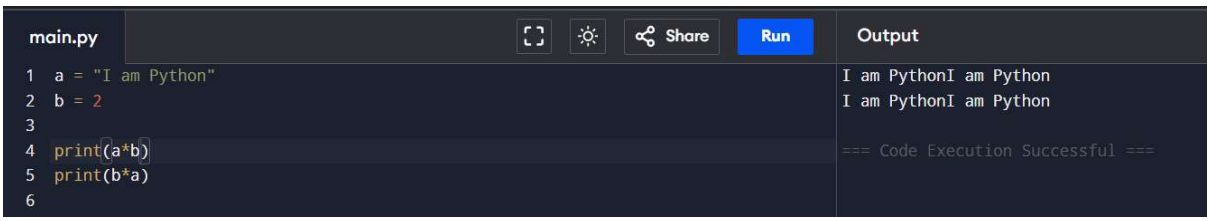
But Floor division (//) can perform both floating point and integral arithmetic. If arguments are int type then result is int type. If atleast one argument is float type then result is float type.

Example:
a = 10
b = 2
print(a/b) → 5.0
print(a//b) → 5

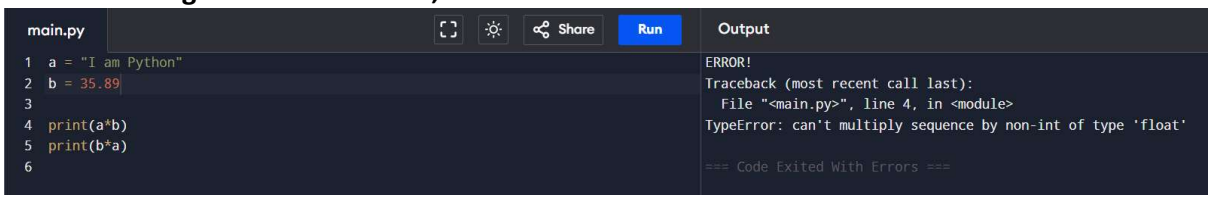
a = 10.0
b = 2
print(a/b) → 5.0
print(a//b) → 5.0

Notes:

- **+ operator, We can use for addition as well as concatenation.**
- **Print (10+20) → Addition → 30**
- **Print("10"+"20") → Concatenation → 1020**
- **Print("Sachin" + "Tendulkar") → Concatenation → SachinTendulkar**
- **Print("Python" + " Java" → Concatenation → Python Java**
- **Print("Python"+57) → It will throw error. Both Data should be either integer or string otherwise error.**
- *** operator, We can use for multiplication as well as string multiplication.**



Floating value not allowed, it will throw error.



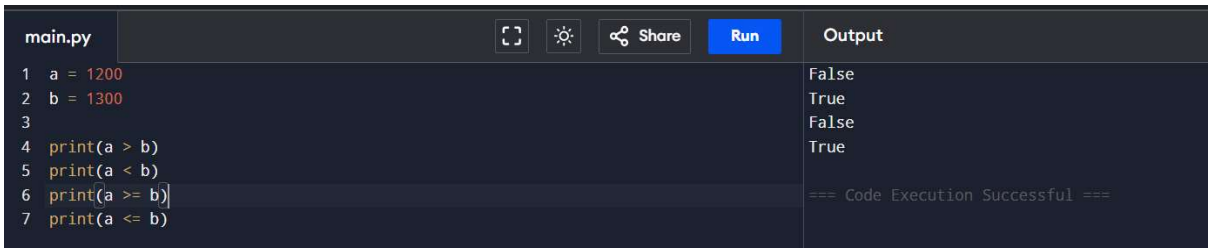
Relational Operators

Comparison or Relational operators compares values. It either returns **True or False** according to the condition.

>, >=, <, <=

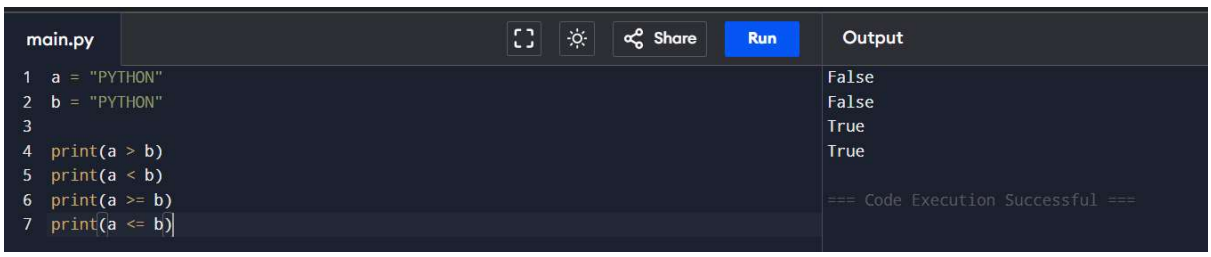
Example 1:-

a = 1200
b = 1300
print(a > b)
print(a < b)
print(a >= b)
print(a <= b)



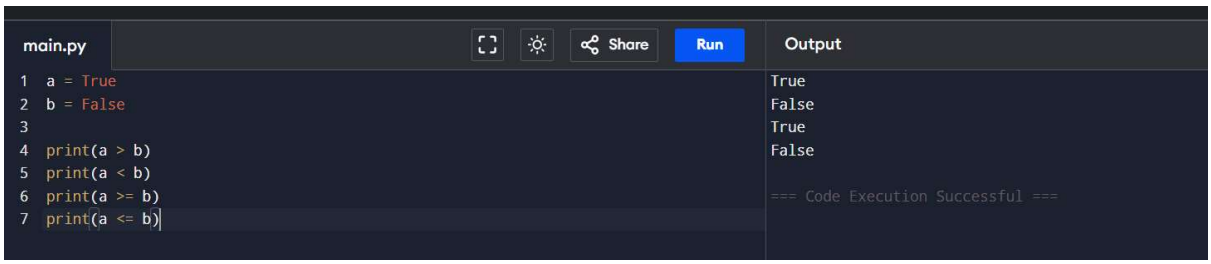
Example 2:-

a = “PYTHON”
b = “PYTHON”
print(a > b)
print(a < b)
print(a >= b)
print(a <= b)



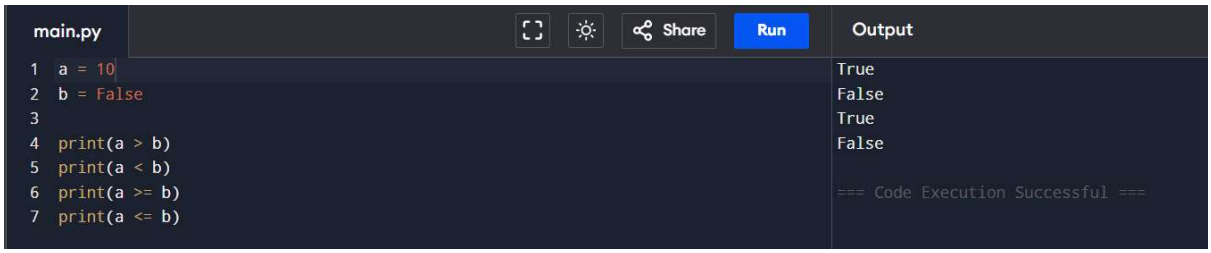
Example 3 :-

a = True
b = False
print(a > b)
print(a < b)
print(a >= b)
print(a <= b)



Example 4 :-

a = 10
b = False ----------→ Python will internally converts this to 0.
print(a > b)
print(a < b)
print(a >= b)
print(a <= b)



Example 5 :-

a = 10
b = “Python”
print(a > b)
print(a < b)
print(a >= b)
print(a <= b)

Not allowed

Equality Operators

==, !=

Example 1:

x = 10
y = 5

Equality check
print(x == 10)----------→ Basically this is content checking.

# Inequality check
print(x != y)

# Other comparisons
print(x > y)
print(y <= 5)

```python
x = 10
y = 5

# Equality check
print(x == 10)

# Inequality check
print(x != y)

# Other comparisons
print(x > y)
print(y <= 5)
```

```
True
True
True
True

=== Code Execution Successful ===
```

Logical Operators

and, or, not

Rules:→

For boolean Types Behaviour:

and → If both arguments are True then only result is True
or → If atleast one arugemnt is True then result is True
not → Complement

For non-boolean Types Behaviour:

0 means False
non-zero means True
empty string is always treated as False

x and y:

If x is evaluates to False return x otherwise return y

x or y:

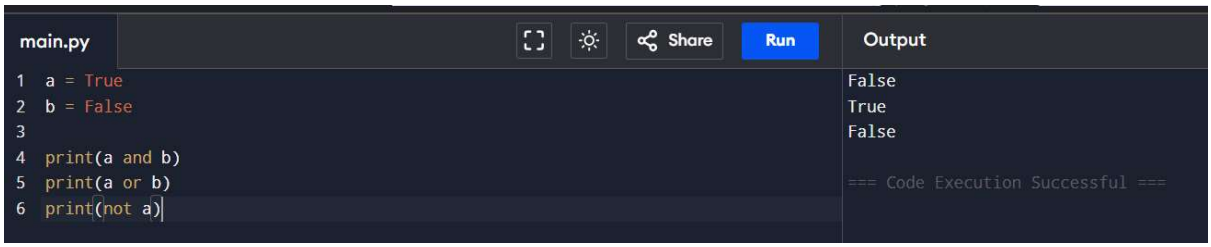
If x evaluates to True then result is x otherwise result is y

not x:

If x is evaluates to False then result is True otherwise False

Example 1:

```
a = True
b = False

print(a and b)
print(a or b)
print(not a)
```



Example 2:

```
print(10 and 20)
print("" and "")
print("" and 10)
print(23.56 and 78.09)
print(10 or 20)
print("" or "")
print(23.56 or 78.09)
print("python" and "java")
print("python" or "java")
print(100 and 456)
print(100 or 456)
print("" and "Python")
print("python" and "")
print("" or "Python")
print("python" or "")
print(not "")
print(not "Python")
```

main.py

```python
print(10 and 20)
print("" and "")
print("" and 10)
print(23.56 and 78.09)
print(10 or 20)
print("" or "")
print(23.56 or 78.09)
print("python" and "java")
print("python" or "java")
print(100 and 456)
print(100 or 456)
print("" and "Python")
print("python" and "")
print("" or "Python")
print("python" or "")
print(not "")
print(not "Python")
```

Output

```
1 20

78.09
10

23.56
java
python
456
100

Python
python
True
False

=== Code Execution Successful ===
```

Bitwise Operators

&, |, ^, ~, <<, >>

& → And
| → OR
^ → Bitwise XOR
~ → Tilde or Bitwise not
<< → Left shift Operator
>> → Right shift operator

- **operators are applicable only for int and boolean types.**

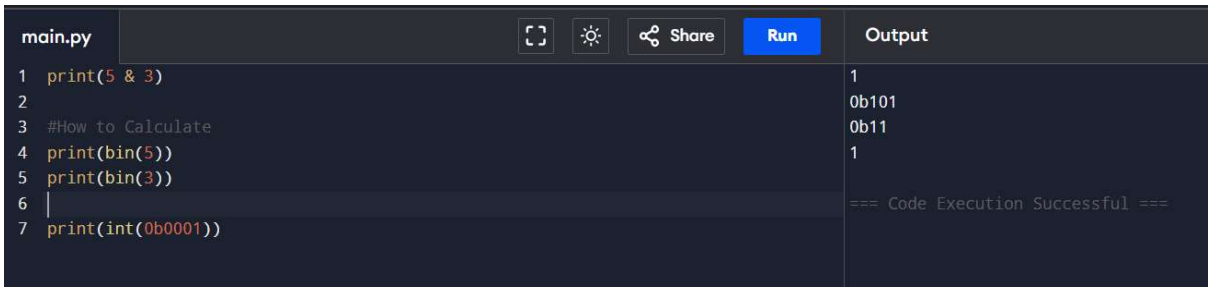
Example 1:

Print(5 & 3) -> output = 1

Calculate first bin(5) then bin(3)
5 -> 0101
3 -> 0011

& -> 0001

Convert 0001 in integer, we have int() function
Int(0b0001) ----------→ 1 is Answer



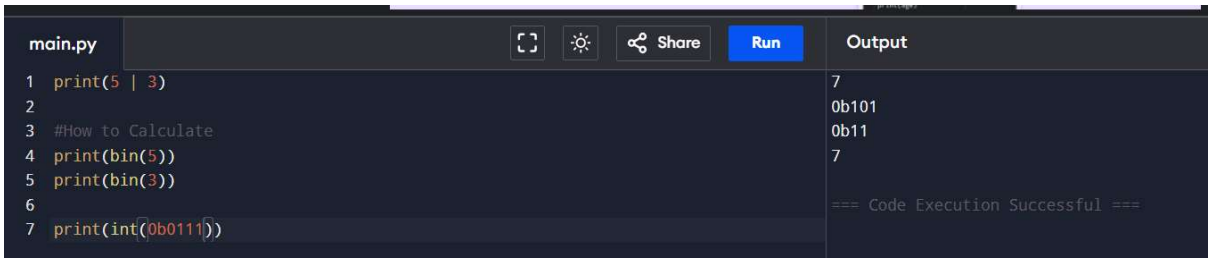
Example 2:

Print(5 | 3) -> output = 7

Calculate first bin(5) then bin(3)
5 -> 0101
3 -> 0011

| -> 0111

Convert 0111 in integer, we have int() function
Int(0b0111) ----------→ 7 is Answer



Example 3:

Print(5 ^ 3) -> output = 6

Calculate first bin(5) then bin(3)
5 -> 0101
3 -> 0011
---
^ -> 0110

Convert 0111 in integer, we have int() function
Int(0b0110) ----------→ 6 is Answer

```
main.py                                   Output
print(5 ^ 3)                              6
                                          0b101
#How to Calculate                         0b11
print(bin(5))                             6
print(bin(3))
                                          === Code Execution Successful ===
print(int(0b0110))
```

Example 4:

Print(~5)

```
main.py                                   Output
print(~5)                                 -6
                                          === Code Execution Successful ===
```

Output = -(value+1) = -6

Example 5:

Print(5 << 1)
Left shift

```
main.py
1 print(5 << 1)
```

```
Output
10

=== Code Execution Successful ===
```

bin of 5 -> 0000 0101
Shift left by 1 position [remove 1 zero from left and shift remaining digits left side, add one zero to complete 8 block]

0	0	0	0	0	1	0	1
After Left shift							
0	0	0	0	1	0	1	0

int(0b00001010)

```
main.py
1 print(int(0b00001010))
```

```
Output
10

=== Code Execution Successful ===
```

Example 6:

print(10 >>2)
Right shift

```
main.py
1 print(10 >>2)
```

```
Output
2

=== Code Execution Successful ===
```

Bin (10) → 0000 1010

0	0	0	0	1	0	1	0

Position is 2 so remove 1, 0 from right…..shift remaing digits to right…..now left two blocks will be empty…..add 0, 0.

0	0	0	0	0	0	1	0

int(0b00000010)

Answer is 2

```
main.py
1  print(int(0b00000010))
```

```
Output
2

=== Code Execution
```

Example 7:

print(10.5 & 5.6)

```
main.py
1  print(10.5 & 5.6)
```

```
Output
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 1, in <module>
TypeError: unsupported operand type(s) for &: 'float' and 'float'

=== Code Exited With Errors ===
```

Assignment Operators

We can use assignment operator to assign value to the variable.

X = 10

A = 56.78

Example:

a = 10

b = a

print(b) → 10

b += a i:e [This one written as **compound assignment**, same as b = b+a]

print(b) → b = b+a →20

b -= a

print(b) → b = b-a →10

b *= a

print(b) → b = b*a → 100

b <<= a

print(b) → b = b<<a →100<<10 → 102400

bin (100) = 01100100

TRICK FOR LEFT AND RIGHT SHIFT OPERATOR

LEFT SHIFT

A much faster way to solve this is using the mathematical shortcut for **left shifts**. Shifting a number left by bits is equivalent to:

$$x \times 2^n$$

Now let's go with some example:

(a) print(10<<1)
2 power 1 -> 2
10*2 = 20

(b) print(10<<2)
2 power 2 → 4
10*4 = 40

(c) print(5<<3)
2 power 3 → 8
5*8 = 40

RIGHT SHIFT

x>>2

$x/2^{(position)}$

Division is considered as floor.

print(10>>1)

2 power 1 -> 2

10/2 = 5 , floor division will give value 5 because both are int value.

print(10>>2)

2 power 2 -> 4

10/4 -> 5.25

floor division value = 5

print(12>>2)

2 power 2 -> 4

12/4 = 3 {floor division}

print(40>>3)

2 power 3 = 8

40/8 = 5 {floor division}

Ternary Operator OR Conditional Operator

x = firstValue if condition else secondValue
If condition is True then firstValue will be considered else secondValue will be considered.

```
a,b=10,20

x=30 if a<b else 40

print(x)
```

```
a,b=10,20
x=30 if a<b else 40
print(x)
```

```
30

=== Code Execution Successful ===
```

Special Operators:

Python defines the following 2 special operators
1) Identity Operators
2) Membership operators

Identity Operators

We can use identity operators for address comparison.
There are 2 identity operators are available
1) is
2) is not

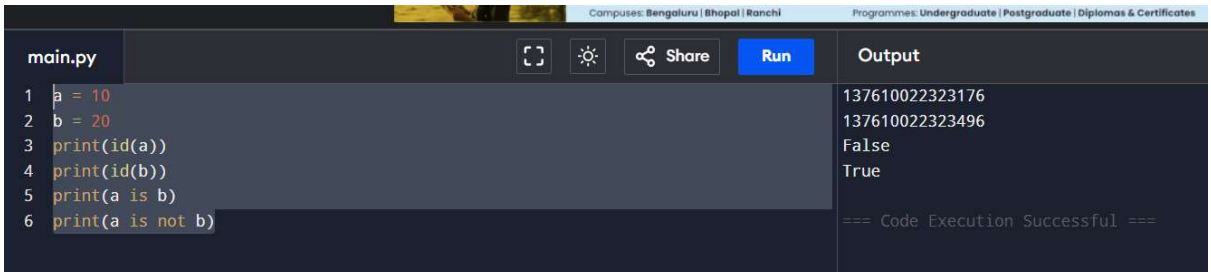
Example:
a = 10
b = 10
print(id(a))
print(id(b))
print(a is b)
print(a is not b)

```
a = 10
b = 10
print(id(a))
print(id(b))
print(a is b)
print(a is not b)
```

```
132596379788264
132596379788264
True
False

=== Code Execution Successful ===
```

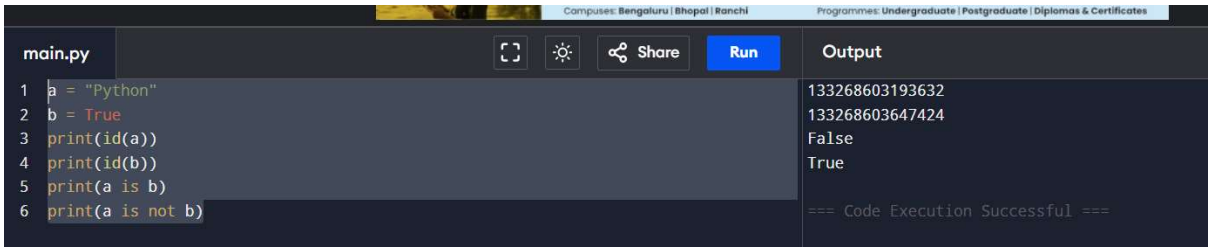
Example

```
a = 10
b = 20
print(id(a))
print(id(b))
print(a is b)
print(a is not b)
```



Example

```
a = "Python"
b = True
print(id(a))
print(id(b))
print(a is b)
print(a is not b)
```



Membership operators

We can use Membership operators to check whether the given object present in the given collection. (It may be String, List, Set, Tuple OR Dict)

In → Returns True if the given object present in the specified Collection
not in → Returns True if the given object not present in the specified Collection

```
x="hello learning Python is very easy"

print('h' in x)

print('d' in x)

print('d' not in x)

print('Python' in x)
```

Example

```
list1=["python","java","cobol","javascript", "typescript"]

print("typescript" in list1)

print("python" in list1)

print("c++" not in list1)

print("cobol" not in list1)
```

```python
list1=["python","java","cobol","javascript", "typescript"]
print("typescript" in list1)
print("python" in list1)
print("c++" not in list1)
print("cobol" not in list1)
```

```
True
True
True
False

=== Code Execution Successful ===
```

Interview Question

1. **What is the difference between `/` and `//` in Python?**
2. **What is the difference between `= =` and `is?`**
3. **What are membership operators?**
4. **What are identity operators?**
5. **What is the output of the expression `(5 > 3) and (2 < 4)?`**
6. **`x = x + 3, How to write in compound assignment way?`**